

Exploratory Test Plan: ParaBank Application

Methodology: Risk-Based Testing

For this exploratory testing, I decided to use Risk-Based Testing to optimize the testing effort. Usually exploratory testing is time-boxed and since we have an application with several functionalities, we want to prioritize features based on the business value.

1. Scope and Focus Areas

The rationale followed in the prioritization below considers high-risk transaction areas: a user being unable to use the application is bad, but it would be even worse if usage impacted their bank account with incorrect debits or credits.

High Priority: Transaction Logic

- **Focus:** *Transfer Funds, Bill Pay, Request Loan.*
- **Strategy:** Validate flows involving transactions. We focus on how the system handles these operations. Besides the happy paths we should check invalid scenarios (such as negative amount), edge scenarios (minimum & maximum amount allowed), and other form validations.

Medium Priority: App access

Even if in the first section we are required to log in with an account in order to check transaction logic, in this section we would try additional scenarios besides a log in happy path.

- **Focus:** *Register and Login.*
- **Strategy:** Verify both login and register flows. We should check we can't register with wrong data (form validation in different fields), with existing users username, and also verify we allow non-ASCII characters both in user and password (I.e.: username is Iñigo). Also, we should validate we actually log in in our account, and not someone's account (yes, this happens! Recently happened on Steam platform).

Low Priority: Account information

It's worth mentioning this is also a vital part of our bank application, but compared with the above sections it is less critical from my point of view. That said, and thinking of a time-boxed, risk based testing, I would leave this as low priority.

- **Focus:** *Account Overview, Transaction Details.*

- **Strategy:** Visual and data consistency checks. We can ensure the history reflects the actions done in the High Priority tier.

2. Technical Risks to Investigate

I'll try to list here some generic technical risks, not oriented into a specific scenario but more on the app itself:

- **Concurrency:** Handling simultaneous requests (I.e.: rapid clicks on "Transfer", different tabs).
- **Security:** How we log in, and what data is visible (I.e.: are we sending sensitive data not encrypted on requests?).
- **Forms and Inputs:** App UI should never let us send incorrect data. We should validate several fields and test boundary limits and edge scenarios. That said, and as a side note, backend logic should always validate requests, as we can always temper with the FE components or even make REST calls ourselves observing how the bank works.

3. Prioritization

As a summary, even if rationale was already explained above, the prioritization matrix would look like this:

- Transfer Funds - Critical – Direct impact in user bank account balance
- Bill Pay – Critical - Impact in user and other customers bank account balance
- Request Loan – Critical - Direct impact in user bank account balance
- Login / Register – High – Essential for access
- Account Overview – Medium – Read-only information
- Transaction Details – Medium – Read-only information
- About us / Static pages – Low – Not important for most common user flows